1 Evaluation Methodology

This chapter details the methodology used to evaluate the Agentic Project Management (APM) framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants. The study compares APM, a multi-agent framework that distributes work across specialized agents, against GitHub Spec-kit, a single-agent toolkit that guides one AI agent through a structured specification-driven workflow. Both frameworks are described in Section 1.2. The case study was conducted in May 2026; all Copilot platform, pricing, and model-availability statements in this chapter refer to that snapshot.

The evaluation measures three distinct dimensions:

- Economic Efficiency (share of the Copilot monthly request allowance consumed per session)
- Process Efficiency (human cognitive load and interaction burden)
- Functional Efficacy (ability to complete non-trivial tasks)

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Planning and Implementation phases) yields a tangible return on investment compared to a single-agent workflow. Prior empirical work on AI coding assistants has concentrated on productivity and task-completion time [2, 6–9], with comparatively little systematic treatment of cost under the subscription-based procurement model through which Copilot is predominantly adopted. This evaluation contributes a cost perspective alongside the process and functional dimensions.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):**
  - Under GitHub Copilot's subscription-plus-quota pricing structure, does APM's multi-agent coordination consume fewer premium requests per session than Spec-kit's single-agent workflow?
  - If a difference in request consumption is observed, is it large enough to meaningfully shift the share of a buyer's monthly allowance burned per task completion?
  - Where in each framework's workflow (planning phase versus implementation phase) does request consumption concentrate, and does that concentration pattern differ between frameworks?
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the amount of manual effort the user must invest during execution, particularly interventions, corrections, and clarifications?

  - If a difference in user effort is observed, is it significant enough to justify the structural overhead APM introduces?
  - Given that the User remains in the loop as a message carrier, can the agents coordinate effectively enough that the User's role stays limited to shuttling messages rather than correcting or redirecting agent behavior?

- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where single-agent workflows are prone to failure due to context exhaustion, loss of coherence across long sessions, or inability to coordinate work across multiple concerns?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Frameworks Under Evaluation

The two frameworks are both open-source and spec-driven, running on the same AI IDE platform. Both structure work through explicit specification and planning artifacts before implementation begins. They differ in how execution is organized: Spec-kit operates within a single agent conversation, while APM distributes work across multiple specialized agents.

1.2.1 GitHub Spec-kit

Spec-kit[1] is a specification-driven development toolkit developed by GitHub. It guides a single AI agent through a structured workflow using three sequential commands:

1. `/speckit.specify`: transforms a feature description into a structured specification containing requirements, user stories, and acceptance criteria.
2. `/speckit.plan`: converts the specification into a technical implementation plan with architecture decisions, data models, API contracts, and supporting documents.
3. `/speckit.tasks`: derives an executable task list (`tasks.md`) from the plan, with parallelism annotations and sequencing constraints.

Execution is triggered by a fourth command, `/speckit.implement`, which parses the task list and carries out each task in sequence. Additional commands support iterative refinement (`/speckit.clarify`), cross-artifact consistency checking (`/speckit.analyze`), and custom quality checklists (`/speckit.checklist`). A constitution mechanism, maintained via the `/speckit.constitution` command, allows users to establish architectural rules that the agent enforces throughout the workflow.

All planning and implementation happens within one continuous conversation. The agent acts as both planner and executor, and the full specification, plan, and task context accumulates in a

[1] Documentation available at: https://github.github.com/spec-kit/

single context window. Spec-kit produces a fixed set of artifact types (specification, plan, data model, contracts, research, tasks) regardless of project type; the templates are predefined and every project receives the same artifact layout.

1.2.2 APM in Comparison

Where Spec-kit runs the entire workflow in a single conversation, APM (described in detail in the preceding chapters) separates planning, coordination, and execution across three agent roles, each operating in its own conversation with scoped context. Agents communicate through a file-based Message Bus rather than a shared context window; planning documents adapt to each project rather than conforming to a predetermined artifact layout; and the user acts as a message carrier between conversations. When an agent's context fills, a Handoff mechanism preserves working knowledge across context boundaries.

1.3 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`,[2] a Python-based Model Context Protocol (MCP)[3] server designed to interface with the `openeclass` platform.[4] Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments. The case study's own material is organized across two repositories, a public task repository and a restricted-access dataset repository for submitted participant data; both are documented in the evaluation appendix.

1.3.1 Task Description

The task requires participants to implement two new announcement tools in the Python MCP server and to create a complete native C replica of the entire server.[5] The two tools are:

- `get_course_announcements`: retrieves announcements for a specific course.
- `get_general_announcements`: aggregates system-wide and per-course announcements.

The C implementation adds significant complexity: participants must manage code translation, memory safety, and protocol details across two very different languages. This combination of feature implementation and full-language replication is calibrated to stress-test the limits of agent coordination and expose any differences between the two frameworks, especially when context management becomes a bottleneck.

Key technical challenges:

[2]Repository available at: https://github.com/sdi2200262/eclass-mcp-server
[3]Protocol specification available at: https://modelcontextprotocol.io
[4]Repository available at: https://github.com/gunet/openeclass
[5]Task specification and grading infrastructure available at:
https://github.com/sdi2200262/bsc-apm-case-study-task

1. Dynamic Data Source: the data required for the new features is not served as static HTML. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.

2. Response Post-Processing: the endpoint returns JSON whose payload is an array of raw HTML strings, so agents must add a second layer of parsing to extract each record; the legacy API also offers no server-side date filtering, so agents must parse and filter locale-formatted dates client-side.

3. C Implementation: agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON or Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing was required. A working Makefile must build the C binary.

1.3.2 Grading

Submissions are evaluated by an automated black-box test suite that applies each participant's patch against fixed baseline commits of the upstream repositories and runs a comprehensive test battery.[6] The test suite awards 100 points total: 12 points for baseline functionality (existing Python tools), 36 points for the two new Python features (18 points each), and 52 points for the C replica implementation. A submission with no patch applied receives the baseline score of 12/100 points.

The grader runs these tests against a Docker-based replica of the openeclass platform rather than against the live University instance that participants develop against. Baseline commits, the replica environment, the multi-stage grading pipeline, and the test-suite composition are documented in the evaluation appendix.

1.4 Study Design

The study is conducted on a single AI IDE platform, VS Code with GitHub Copilot, and compares the two frameworks across a homogeneous cohort of participants. It is designed as a within-subjects experiment, so that each participant contributes data points for both frameworks and per-participant variance in skill, prior experience, and working style is absorbed into each pairwise comparison rather than confounding it. The single-platform, single-model design mirrors empirical enterprise adoption patterns: Demirer et al. [3] report that more than 90% of commercial LLM API workload concentrates on a single chosen model even when multi-homing is available.

Participants are drawn from a cohort of University students and working professionals with active access to GitHub Copilot, either through the GitHub Education Pack or through a standard premium subscription.

[6] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.4.1 Experimental Design

Each participant completes the case study task twice: once with APM, once with Spec-kit. Each session is time-boxed to a maximum of three hours, after which the participant stops regardless of completion state. The order of the two frameworks is counterbalanced across participants, so that approximately half of the cohort encounters APM first and the other half encounters Spec-kit first. This counterbalancing lets us attribute any observed first-to-second session improvement to task familiarity rather than to the framework itself, and is analyzed explicitly under Learning Effects in Section 2.

The two sessions are isolated from one another using Git, so no code, state, or context from the first session can influence the second. Session isolation, a branch-and-patch workflow, and per-session submission packaging are documented in the evaluation appendix.

1.4.2 Data Source

Each session produces three artifacts that together form the participant submission: the `.patch` file generated from the session's isolated Git branch, the set of JSON chat exports produced by Copilot for every conversation opened during the session, and a post-session questionnaire response (described in Section 1.4.3). The chat exports provide structured records of every conversation (model identifiers, message timestamps, tool invocations, and per-message speaker roles), enabling programmatic extraction of both request-level and interaction-level metrics without manual annotation.

Because GitHub Copilot is billed per seat with a monthly premium-request allowance and a marginal per-request charge for overage consumption, the per-conversation request counts preserved in the export metadata correspond directly to the unit in which the buyer's cost is incurred.[7] The Copilot procurement model (three-part tariff, plan allowances, and model multipliers) is described in the evaluation appendix.

APM's multi-agent workflow spans multiple Copilot conversations within a single session (one for the Planner, one for the Manager, and multiple for the Workers), while Spec-kit operates within a single conversation. To keep the two frameworks comparable, all metrics are computed by aggregating every conversation file belonging to a participant submission into a unified session-level view. Participant data is anonymized and encrypted before storage; the sanitization scheme is documented in the evaluation appendix.

1.4.3 Post-Session Questionnaire

Each session concludes with a short post-session questionnaire that captures self-reported measures complementing the chat-export-derived metrics. The instrument records the participant's completion state (whether the three-hour cap was hit, the task was completed earlier, or the session was stopped early), prior familiarity with the framework, confidence in the submitted solution, and perceived framework effectiveness. It additionally captures five workload items

[7] Premium-request billing structure and overage rates documented at: https://docs.github.com/en/billing/concepts/product-billing/github-copilot-premium-requests

adapted from the NASA Task Load Index, an established instrument for subjective workload assessment: mental demand, effort, frustration, verification burden, and prompt-engineering burden. In the second session, participants additionally report whether completing the task with the second framework felt easier, harder, or the same compared to the first. All scored items use a five-point Likert scale. Two optional free-text fields allow participants to elaborate on the comparison and provide general feedback. The questionnaire is submitted alongside the per-session data upload through the submission pipeline documented in the evaluation appendix.

1.5 Metrics and Data Collection

The metrics are organized into three measurement dimensions (economic, process, and functional), followed by the statistical analysis plan that applies them. The subsections below specify each in turn.

1.5.1 Economic

The primary economic metric is **premium requests per session**: the total number of premium requests consumed across all conversations of a single framework session. Each participant contributes one APM-session count and one Spec-kit-session count for the same task, compared pairwise across the cohort. The metric maps directly onto the buyer's monthly-allowance exposure under Copilot's subscription-plus-quota procurement model. Lambrecht et al. [4] show empirically that under this pricing structure, buyer tariff choice is driven by demand uncertainty rather than marginal price, with consumers steering toward allowance-based plans to hedge against bill variance, and Li and Kumar [5] identify predictable operating cost as a primary driver of SaaS pricing structure; the share of allowance consumed is therefore the buyer-relevant cost signal for subscription customers.

The secondary economic metric reports the same per-session count broken down by phase. Planning-phase and implementation-phase boundaries are detected automatically per framework; detection rules are documented in the evaluation appendix. The phase split locates where each framework spends its budget and exposes the source of any observed cost differential.

All sessions use a single underlying model, held fixed across both frameworks and across both sessions of each participant: GPT-5.4. This is a 1× premium-request model, available on every plan tier relevant to the participant pool. Holding the model fixed at a uniform 1× multiplier eliminates multiplier-induced variance, so that a difference in premium-request counts reflects a difference in framework behavior rather than a difference in model choice. The Copilot pricing structure, plan allowances, and multiplier mechanics are documented in the evaluation appendix.

The metric measures share of the monthly premium-request allowance consumed within the subscription-plus-quota procurement model; it does not measure dollar-equivalent inference cost, which is decoupled from the subscription price and has declined by roughly three orders of magnitude over the preceding two years [3]. The subscription buyer is by definition insulated from that translation, so the procurement-model scoping is intentional rather than a defect.

1.5.2 Process

The process metrics capture the coordination burden each framework imposes on the user across a session, together with the reliability with which each framework's agents produce directly usable output. Ziegler et al. [9] identify acceptance of AI-generated output as the strongest observable predictor of perceived developer productivity; the reliability sub-metrics below operationalize acceptance-like signals from the tool-call and code-modification events preserved in the chat export.

Each user message is assigned to one of three buckets: short (brief acknowledgments or routing messages), protocol (framework-protocol messages such as APM handoff routing or Spec-kit slash-command invocations), and standard (substantive task-related prompts). The **behavior distribution** reports the share of each bucket in the session's total user-message count; the **behavior magnitude** reports the average character count per bucket. Together these views expose whether a framework's user burden is concentrated in mechanical message-shuttling (high protocol share, low magnitude) or in substantive problem-solving (high standard share, high magnitude).

Session duration is the wall-clock time from the first user message of the session to the last assistant message, bounded above by the three-hour cap. Sessions that hit the cap and sessions that complete earlier are distinguished using the completion state reported in the post-session questionnaire.

Three sub-metrics characterize the reliability of agent output. The **tool success rate** is the proportion of agent tool invocations that return a non-error result. The **tool retry rate** is the proportion of invocations followed within the same agent turn by a repeated call to the same tool with adjusted arguments, indicating that the first attempt failed or was rejected. The **user edit rate on terminal commands** is the proportion of terminal-execution tool calls whose command string was modified by the user before execution rather than executed verbatim, indicating a lack of trust in the generated command.

1.5.3 Functional

The functional dimension is evaluated against the 100-point grading rubric introduced in Section 1.3.2, which scores baseline preservation (12 points), Python feature correctness (36 points), and C replica correctness (52 points). Prior empirical work on AI coding assistants, including Cui et al. [2] and Paradis et al. [6], typically collapses task completion into a single binary outcome; the partial-credit structure preserves the granularity needed to compare frameworks on complex multi-component work.

1.5.4 Analysis Plan

Each paired framework comparison uses a paired t-test on the participant-level differences, reported alongside Cohen's d, a standardized effect-size estimate that expresses the mean difference between two groups in pooled-standard-deviation units, distinguishing statistical significance from practical significance. The economic, process, and functional metrics are each analyzed

this way: total rubric score, per-category test-category scores, completion rates, premium requests per session, phase-broken request shares, behavior distribution and magnitude per bucket, reliability sub-metrics, and session duration.

Correlations between cost, process, and functional metrics are reported as Pearson coefficients within each framework and across the combined cohort. These surface whether framework-induced variation in cost or process burden translates into functional outcomes.

Learning effects are analyzed separately across the counterbalanced orderings. A learning-effect matrix reports mean first-to-second-session improvement across both framework orderings for each metric; an order-effect comparison between APM-first and Spec-kit-first directions tests whether the order of exposure affects the improvement; and the aggregate first-to-second-session improvement across orderings gives the baseline task-learning estimate.

2 Results

2.1 Economic

[Forthcoming. Quantitative results for the primary metric (premium requests per session) and the secondary phase-broken breakdown. Tables: paired APM-versus-Spec-kit distributions with paired t-test statistics, p-values, and Cohen's d; planning-versus-implementation phase shares per framework; Pearson correlations against rubric score within and across frameworks.]

2.2 Process

[Forthcoming. Quantitative results for the user-burden and reliability metrics. Tables: behavior distribution and magnitude per bucket (short, protocol, standard) paired across frameworks with t-statistics and Cohen's d; tool success rate, tool retry rate, and user edit rate on terminal commands per framework; session-duration distributions against the three-hour cap; Pearson correlations against rubric score. Self-reported workload items from the post-session questionnaire are reported alongside the instrumented coordination metrics.]

2.3 Functional

[Forthcoming. Quantitative results for the 100-point rubric. Tables: total score distribution, completion-rate bands (full 100/100, partial 49–99, partial 12–48, baseline-only 12/100), baseline-regression rates, per-test-category success rates (build, protocol, login, logout, courses, authstatus, announcements) for both the Python and C implementations, and paired framework comparisons with t-statistics and Cohen's d. Perceived framework effectiveness from the post-session questionnaire is reported alongside the rubric outcomes.]

2.4 Learning Effects

[Forthcoming. Quantitative results for the learning-effect analysis: first-to-second-session improvement matrix across framework orderings for functional, economic, and process metrics; order-effect comparison between APM-first and Spec-kit-first directions; and the aggregate baseline task-learning estimate across orderings.]

3 Discussion

3.1 Economic Efficiency

[Forthcoming. Synthesis anchors: Lambrecht et al. [4] on allowance-size dominance over marginal price under demand uncertainty, translating any observed per-session request delta into a shift in the buyer's monthly-allowance exposure; Demirer et al. [3] on the subscription-predictability premium over metered inference and the single-model concentration pattern in enterprise workloads, situating the cost finding within the broader LLM procurement landscape; Li and Kumar [5] on predictable operating cost as the primary driver of SaaS pricing structure, grounding why monthly-allowance exposure is the buyer-relevant cost signal. Observation links: per-session and phase-broken request deltas, share of allowance consumed per framework, and the phase breakdown read against the workflow structures described in Section 1.2.]

3.2 Process Efficiency

[Forthcoming. Synthesis anchors: Becker et al. [1] on the time-allocation breakdown of AI-assisted coding (review, prompt, wait) and the sub-44% acceptance rate, establishing the verification-burden shape to which observed deltas are compared; Paradis et al. [6] on the scaling of verification cost with coder experience, interpreting any across-framework asymmetry in prompt magnitude; Ziegler et al. [9] on acceptance rate as the strongest observable predictor of perceived productivity, connecting the reliability sub-metrics to perceived productivity; Vukovic et al. [8] on multi-file architect reasoning as an unmet enterprise requirement, framing the structural motivation for the multi-agent comparison; Cui et al. [2] on the build-count versus pull-request-count asymmetry, giving a coordination-burden signature against which behavior-distribution shifts are read. Observation links: behavior distribution and magnitude deltas per bucket, reliability sub-metrics (tool success, retry, and user-edit rates), session-duration distributions against the three-hour cap, and self-reported workload items from the post-session questionnaire paired against the instrumented coordination measures.]

3.3 Functional Efficacy

[Forthcoming. Synthesis anchors: Cui et al. [2] on the +26% pull-request-completion baseline observed in enterprise-field conditions, the reference point for cross-framework functional comparison; Peng et al. [7] on the 55.8% time-reduction finding in the isolated-task lab setting, establishing the lab-to-field gradient against which the case-study setting is positioned; Becker

et al. [1] on the 19% slowdown of experienced developers working with AI on mature complex repositories, framing the single-context-window limit case that the testbed task is designed to challenge; Vukovic et al. [8] on multi-file architect reasoning as the top unmet enterprise requirement, connecting observed functional outcomes to the design motivation for multi-agent coordination. Observation links: total rubric score, per-category test-category success rates, completion-rate bands distinguishing full, partial, and baseline-only outcomes, framework-specific failure patterns, and perceived framework effectiveness from the post-session questionnaire reported alongside the rubric outcomes.]

3.4 Threats to Validity

This subsection enumerates internal, external, and construct validity threats and the mitigations applied.

3.4.1 Internal Validity

Learning effects from the within-subjects design. Each participant completes the same task twice, and task familiarity from the first session can influence performance in the second. Counterbalancing the framework order across participants mitigates the framework-effect confound but does not eliminate the underlying learning curve. The Learning Effects analysis in Section 2 quantifies the mean improvement between sessions across both orderings and is the primary defence against this threat.

Self-reported versus observed measures. Subjective developer-reported productivity often diverges from objective time-in-motion measurements, occasionally in the opposite direction [1, 7]. This study pairs the instrumented, export-derived metrics with a post-session questionnaire covering workload, confidence, and perceived framework effectiveness (Section 1.4.3); observed divergences between self-report and instrument are reported as findings rather than absorbed as an unstated limitation. Residual risk remains: self-reports are subject to recall bias and social-desirability pressures that the instrument-side measurements cannot independently correct.

3.4.2 External Validity

Generalization scope. The case study is run on a single AI IDE platform with a single fixed model, against a complex software engineering task requiring two-language implementation and legacy-codebase research, in a setting positioned between lab and field conditions. In an empirical analysis of twelve commercial LLM use-case categories, Demirer et al. [3] report that programming commands both the highest average intelligence index and the highest willingness to pay among buyers, so the testbed sits in the domain most sensitive to marginal AI capability but not in administrative, analytical, or creative workflows. Lab-style productivity measurements for AI coding assistants have exceeded enterprise field measurements by more than a factor of two, and experienced developers on mature codebases have been observed to be slower with AI than without [1, 2, 7]; the case-study measurements are neither pure laboratory benchmarks nor natural-setting observations. Findings generalise within this scope and do not generalise to

multi-platform settings, dynamic cross-model routing, simpler or less knowledge-intensive tasks, or pure laboratory or natural-setting conditions.

Temporal decay of findings. The AI coding assistant field is evolving rapidly. Not only effect sizes but also descriptive claims throughout this chapter (plan allowances, model multipliers, command sets, and framework releases) are tied to the May 2026 case-study date stated in Section 1. Allowance sizes and multipliers are subject to vendor change, and findings that rest on specific pricing parameters are defensible against that snapshot but may require re-estimation under materially different parameters [4].

3.4.3 Construct Validity

Behavioral input buckets as process-efficiency proxy. The short, protocol, and standard input buckets are defined for this study and extracted from the chat-export content. They are not validated against an external coordination-burden instrument such as the SPACE framework [9] or the NASA Task Load Index. Their validity as proxies for human coordination effort rests on face validity alone: the relative ordering of frameworks along these metrics is robust, but the absolute magnitude of differences is interpretive rather than calibrated.

Instrument coverage limits. The 100-point rubric captures correctness and robustness but not code quality, maintainability, security posture, or long-term technical debt; prior empirical work on AI coding assistants has consistently noted that code-quality dimensions are not captured by pass/fail functional measurement [2, 6], and this study inherits that limitation. Metric extraction is further limited to the per-conversation JSON chat exports that accompany each participant submission: inline code-completion events that are not part of a chat conversation are invisible to the metric set, so the reliability and process metrics characterise only the agent-driven portion of the workflow rather than the full IDE interaction.

Grader environment parity. The grading pipeline runs submitted patches against a Docker-based replica of the openeclass platform rather than against the live University of Athens instance that participants develop against. The replica environment (Docker image, fixture data, authentication stubs) is documented in the evaluation appendix. Divergences between the replica and the live instance (for example, timezone handling, localized date parsing, or session-cookie behavior) can in principle cause a patch that works locally to miss points during grading. Fixed baseline commits and deterministic seeded test data mitigate but do not eliminate this risk.

References

[1] Joel Becker, Nate Rush, Beth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity, 2025.

[2] Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. Working paper, February 2025.

[3] Mert Demirer, Andrey Fradkin, Nadav Tadelis, and Sida Peng. The emerging market for

intelligence: Pricing, supply, and demand for LLMs. Working Paper 34608, National Bureau of Economic Research, 2025.

[4] Anja Lambrecht, Katja Seim, and Bernd Skiera. Does uncertainty matter? Consumer behavior under three-part tariffs. *Marketing Science*, 26(5):698–710, 2007. doi: 10.1287/mksc.1070.0283.

[5] Bo Li and Subodha Kumar. Managing software-as-a-service: Pricing and operations. *Production and Operations Management*, 31(6):2588–2608, 2022. doi: 10.1111/poms.13729.

[6] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. How much does AI impact development speed? An enterprise-based randomized controlled trial, 2024.

[7] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot, 2023.

[8] Maja Vukovic, Rangeet Pan, Tin Kam Ho, Rahul Krishna, Raju Pavuluri, and Michele Merler. Usage, effects and requirements for AI coding assistants in the enterprise: An empirical study. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2026. arXiv preprint arXiv:2601.20112v1.

[9] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot's impact on productivity. *Communications of the ACM*, 67(3):54–63, 2024. doi: 10.1145/3633453.